UVa SEGOVIA

inf5G INFORMÁTICA ESCUELA INGENIERÍA SEGOVIA

CAMPUS PÚBLICO MARÍA ZAMBRANO SEGOVIA

Universidad de Valladolid

Lab 4. Clase Timer y animaciones

[DISEÑO DE INTERFACES DE USUARIO]

Víctor Martínez Cagigal

Revisión: Nicolás Sacristán Machín

Contenidos

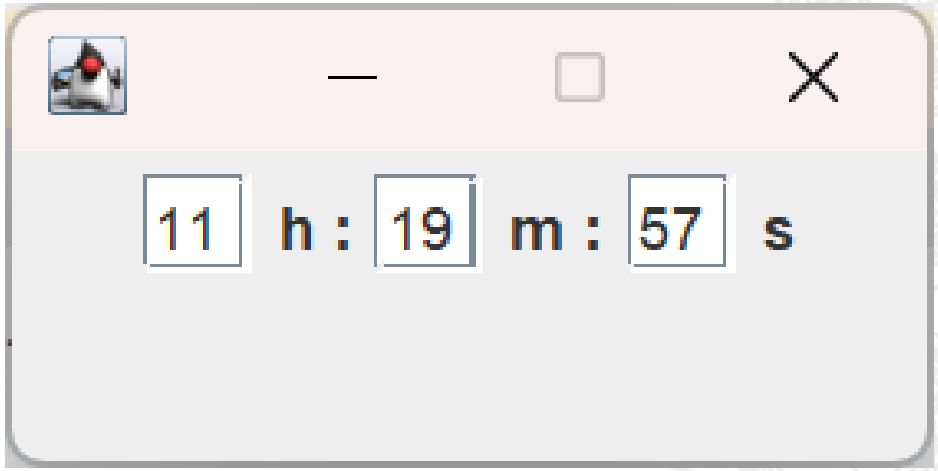
- La clase Timer
- Animaciones en Java Swing

La clase Timer

- Los objetos de la La clase Timer del paquete Swing generan una secuencia de eventos de acción (ActionEvents) equiespaciados en el tiempo.
- `Timer(int delay, ActionListener listener)`
  - **`delay`**: tiempo expresado en milisegundos que representa tanto el tiempo en dispararse el primer evento como el tiempo que tiene lugar entre eventos.
  - **`listener`**: Es el escuchardor encargado de manejar la acción ligada al evento disparado por el objeto Timer.

Ejemplo 1. Un reloj sencillo

- Un objeto de la clase `Timer` en conjunción con uno de la clase `LocalTime` nos permite implementar de una forma sencilla un reloj:



Ejemplo 1. Un reloj sencillo

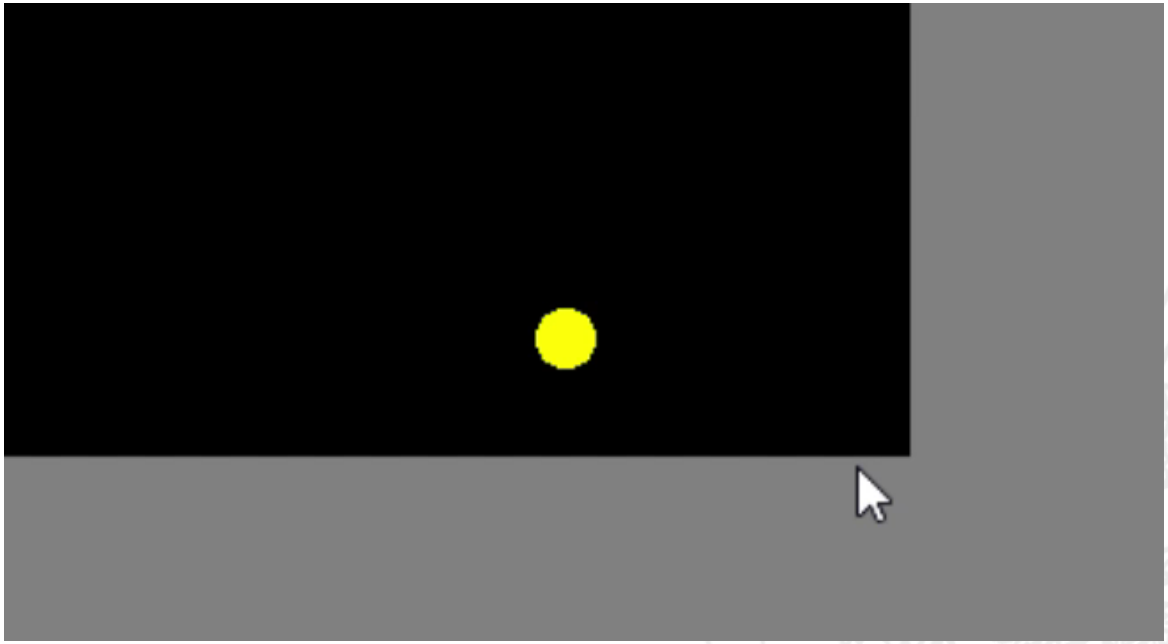
```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.time.LocalTime;
import javax.swing.*;

public class Ej1_Reloj extends JFrame {

    private static final int FIELD_WIDTH = 2;
    private static final int TIMER_INTERVAL = 1000;

    private JTextField hourField = new JTextField(FIELD_WIDTH);
    private JTextField minuteField = new JTextField(FIELD_WIDTH);
    private JTextField secondField = new JTextField(FIELD_WIDTH);
    private JLabel hourLabel = new JLabel("h :");
    private JLabel minuteLabel = new JLabel("m :");
    private JLabel secondLabel = new JLabel("s");
    private Timer timer;

    public Ej1_Reloj() {
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        setSize(200, 100);

        add(hourField);
        add(hourLabel);
        add(minuteField);
        add(minuteLabel);
        add(secondField);
        add(secondLabel);

        timer = new Timer(TIMER_INTERVAL, new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent event) {
                updateClock();
            }
        });

        setVisible(true);
    }

    private void updateClock() {
        LocalTime time = LocalTime.now();
        hourField.setText(formatNumber(time.getHour()));
        minuteField.setText(formatNumber(time.getMinute()));
        secondField.setText(formatNumber(time.getSecond()));
    }

    private String formatNumber(int number) {
        return String.format("%02d", number);
    }

    public void startClock() {
        timer.start();
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new Ej1_Reloj().startClock());
    }
}
```

El objeto ActionListener se asocial con el Timer en el constructor. Al iniciarse el Timer se llamará a la función updateClock() cada segundo

Movimiento de un objeto gráfico

- Animar un objeto gráfico implica pintar y repintar una misma escena de tal forma que genere la ilusión de una imagen en movimiento.
- Para animar un objeto gráfico deberemos:
  - Dibujar el objeto
  - Mover el objeto a una nueva posición.

Ejemplo 2. Pelota con rebote en los límites

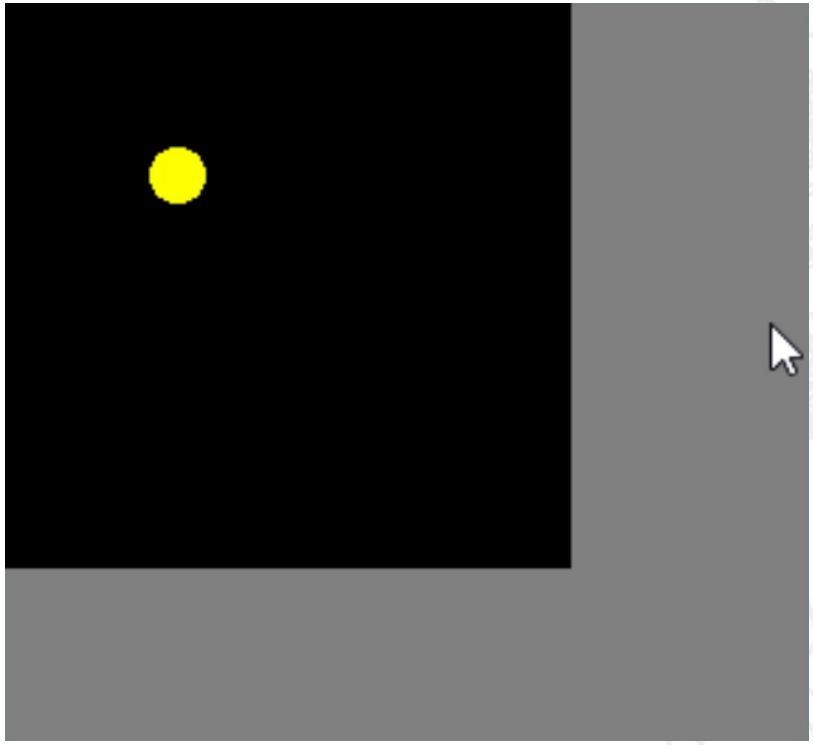


Ejemplo 2. Pelota con rebote en los límites

```java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class Ej2_MovRebote extends JFrame {
    // Constants
    private static final int WINDOW_WIDTH = 400;
    private static final int WINDOW_HEIGHT = 250;
    private static final int STEP_SIZE = 5;
    private static final int TIMER_DELAY = 10;

    private Timer timer;
    private int ballX = 200;
    private int ballY = 100;
    private boolean movingRight = true;
    private ImageIcon ballIcon;
    private int ballWidth, ballHeight;
    private final DrawCanvas canvas;

    public Ej2_MovRebote() {
        // Load the ball image
        ballIcon = new ImageIcon(getClass().getResource("/pacman.gif"));
        ballWidth = ballIcon.getIconWidth();
        ballHeight = ballIcon.getIconHeight();

        canvas = new DrawCanvas();
        add(canvas);

        // JFrame settings
        setTitle("Pelota con Rebote");
        setSize(WINDOW_WIDTH + 20, WINDOW_HEIGHT + 40);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);

        // Set up the timer
        timer = new Timer(TIMER_DELAY, e -> updateBallPosition());
    }

    private void updateBallPosition() {
        if (movingRight) {
            if (ballX < WINDOW_WIDTH - ballWidth - STEP_SIZE) {
                ballX += STEP_SIZE;
            } else {
                movingRight = false;
            }
        } else {
            if (ballX >= STEP_SIZE) {
                ballX -= STEP_SIZE;
            } else {
                movingRight = true;
            }
        }
        repaint();
    }

    public void startAnimation() {
        timer.start();
    }
    private class DrawCanvas extends JPanel {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            setBackground(Color.GRAY);
            g.setColor(Color.BLACK);
            g.fillRect(0, 0, WINDOW_WIDTH, WINDOW_HEIGHT);
            ballIcon.paintIcon(this, g, ballX, ballY);
        }
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new Ej2_MovRebote().startAnimation());
    }
}
```

Ejemplo 3. Animación estática

- Para animar un Sprite de forma estática, basta con cambiar sus imágenes con una frecuencia constante:



Ejemplo 3. Animación estática

```java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class Ej3_AnimEstatica extends JFrame {
    private static final int WINDOW_SIZE = 300;
    private static final int TIMER_DELAY = 100;
    private static final int IMAGE_X = 50;
    private static final int IMAGE_Y = 50;

    private final ImageIcon[] imageArray;
    private int currentImageIndex = 0;
    private final DrawCanvas canvas;
    private final Timer timer;

    public Ej3_AnimEstatica() {
        // Load images
        imageArray = new ImageIcon[] {
                new ImageIcon(getClass().getResource("/pacman.gif")),
                new ImageIcon(getClass().getResource("/right1.gif")),
                new ImageIcon(getClass().getResource("/right2.gif")),
                new ImageIcon(getClass().getResource("/right3.gif"))
        };

        // JFrame settings
        setTitle("Static Animation");
        setAlwaysOnTop(true);
        setSize(WINDOW_SIZE, WINDOW_SIZE);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);

        // Set up canvas
        canvas = new DrawCanvas();
        add(canvas, BorderLayout.CENTER);

        // Set up timer
        timer = new Timer(TIMER_DELAY, e -> updateAnimation());
    }

    private void updateAnimation() {
        currentImageIndex = (currentImageIndex + 1) % imageArray.length;
        repaint();
    }

    public void startAnimation() {
        timer.start();
    }

    private class DrawCanvas extends JPanel {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            setBackground(Color.GRAY);
            g.setColor(Color.BLACK);
            g.fillRect(0, 0, WINDOW_SIZE, WINDOW_SIZE);
            imageArray[currentImageIndex].paintIcon(this, g, IMAGE_X, IMAGE_Y);
        }
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new Ej3_AnimEstatica().startAnimation());
    }
}
```

PRÁCTICA 4. TIMERS Y ANIMACIONES

Desarrolla una aplicación Swing que combine los conceptos vistos en los ejemplos anteriores. La aplicación mostrará un sprite animado (Pac-Man) que se mueve por el canvas rebotando en los cuatro bordes, con las siguientes características:

1. **Movimiento en dos ejes.** El sprite se mueve simultáneamente en horizontal y vertical, rebotando al alcanzar cualquiera de los cuatro bordes del canvas.
2. **Animación del sprite.** Mientras se mueve, el sprite alterna entre sus imágenes de animación de forma que parezca que está "caminando".
3. **Control de la animación.** Añade dos botones **"Iniciar"** y **"Pausar"** que arranquen y detengan la animación usando los métodos start() y stop() del Timer.
4. **Visualización de coordenadas.** Muestra en un Jlabel las coordenadas actuales del sprite (x, y), actualizándose en cada tick del Timer.
5. **Control de velocidad.** Añade un Jslider que permita al usuario ajustar la velocidad del movimiento en tiempo real, modificando el valor de STEP_SIZE entre 1 y 20.
6. **Orientación del sprite.** Haz que el sprite cambie de imagen según la dirección en la que se está moviendo: una imagen distinta para moverse hacia la derecha y otra para moverse hacia la izquierda. Si tienes imágenes para arriba y abajo, úsalas también.
7. **Rastro de movimiento.** Guarda en un ArrayList las últimas 10 posiciones del sprite y dibújalas en el canvas como una estela de círculos de tamaño y opacidad decrecientes, de forma que se aprecie la trayectoria reciente.